VulMorph-Fed: Cross-Project Software Vulnerability Detection via Federated Vulnerability Morphology Learning

Anonymous Authors

Abstract

Deep learning-based software vulnerability detection faces two coupled barriers: models fail to generalise across projects because they overfit project-specific lexical patterns, and organisations cannot pool the proprietary code that would diversify training data. We propose VulMorph-Fed, a federated learning framework built on the observation that vulnerability manifestations vary lexically across projects while their structural characteristics (morphologies) are largely shared. VulMorph-Fed combines three components: Vulnerability-Critical Subgraph Abstraction (VCSA), which maps code tokens onto a small, fully specified taxonomy of abstract operation types and learns to down-weight vulnerability-irrelevant edges; Morphology-Conditioned Federated Prototype Aggregation (MCFPA), which shares CWE-conditioned prototypes instead of model parameters under a provable per-round ε-differential-privacy guarantee with explicit composition accounting; and Morphology-Guided Message Passing (MGMP), which injects the aggregated global prototypes into local graph neural network training. We evaluate on project-level cross-project splits of four public C/C++ corpora (Devign, BigVul, DiverseVul, PrimeVul) against baselines organised by privacy class and matched in data, budget, backbone and input representation, reporting mean and standard deviation over repeated runs with statistical tests. At matched per-round privacy budgets, VulMorph-Fed substantially outperforms DP-FedAvg — whose parameter-level noise collapses detection to near-chance — while retaining most of the discrimination of non-private federated training, and communicating only $\sim$11 KB per client per round. The abstracted repre-

sentation itself transfers: parameter-averaging FL on our morphology graphs surpasses its lexical counterparts on the multi-project corpora. The complete implementation and scripts regenerating every reported number are publicly released.

Keywords: Software Vulnerability Detection, Federated Learning, Graph Neural Networks, Cross-Project Generalisation

1. Introduction

Software vulnerabilities are a critical and growing threat to modern computing systems. The National Vulnerability Database (NVD) reported over 25,000 new Common Vulnerabilities and Exposures (CVEs) in 2024 alone, highlighting the urgent need for automated and scalable vulnerability detection mechanisms. While Deep Learning-based Vulnerability Detection (DLVD) methods have demonstrated significant promise, they continue to suffer from two fundamental barriers that impede their adoption in real-world scenarios: the generalisation gap and the data silo problem.

First, state-of-the-art DLVD models exhibit a catastrophic *generalisation gap* when applied across different software projects [1]. For instance, a model trained on the source code of FFmpeg typically fails to reliably detect vulnerabilities in OpenSSL, even when both projects manifest the identical Common Weakness Enumeration (CWE) type. Replication studies consistently report performance drops of 50% to 90% in cross-project evaluation settings compared to within-project evaluations [2, 3]. This degradation occurs because existing models tend to overfit to project-specific lexical patterns (e.g., API names, variable naming conventions) rather than learning the underlying structural semantics of the vulnerability.

Second, the *data silo problem* prevents the aggregation of sufficient training data. Real-world vulnerability data is highly sensitive and routinely proprietary. Organisations such as financial institutions, cloud service providers, and defence contractors cannot legally or practically share their raw source code with

external entities due to intellectual property concerns and privacy regulations. Consequently, a centralised DLVD model trained on one organisation's codebase cannot leverage the knowledge embedded in another organisation's repositories, severely restricting the model's exposure to diverse vulnerability manifestations.

Federated Learning (FL) has emerged as a promising paradigm to address the data silo problem by enabling collaborative model training without centralising raw data [4, 5]. However, naively applying standard FL techniques, such as FedAvg, to vulnerability detection does not resolve the generalisation gap [6]. Weight averaging across highly heterogeneous, non-IID (Independent and Identically Distributed) codebases fails to transfer structural vulnerability knowledge effectively, leading to suboptimal cross-project detection performance [7, 8].

The root cause of both barriers stems from the representation level: existing approaches primarily model code at the lexical or token level rather than at the structural or semantic level [9, 10]. A buffer overflow in FFmpeg and a buffer overflow in OpenSSL are structurally identical within their Code Property Graphs (CPGs)—both typically involve a loop incrementing an index, an array access using that index, and a missing bounds check—yet they utilise completely different function names and API calls.

To overcome these challenges, we propose **VulMorph-Fed** (Vulnerability Morphology-Guided Federated Learning), a novel federated learning framework designed to achieve robust cross-project vulnerability detection while preserving data privacy. VulMorph-Fed operates on the principle that vulnerability patterns are structurally invariant across projects but lexically heterogeneous. By extracting and federating *vulnerability morphologies* rather than raw parameters or lexical features, VulMorph-Fed successfully bridges the generalisation gap without requiring centralised code sharing.

The primary contributions of this paper are:

- We introduce the **Vulnerability-Critical Subgraph Abstraction (VCSA)**, which combines a fully specified, deterministic mapping of code tokens

onto a small taxonomy of abstract semantic operation types (given exhaustively in Table 1) with a learned, trainable edge-weighting module that down-weights vulnerability-irrelevant structure.

- We propose **Morphology-Conditioned Federated Prototype Aggregation (MCFPA)**, an FL aggregation mechanism that shares CWE-conditioned semantic prototypes instead of model parameters. We prove a per-round ε-differential-privacy guarantee for the released prototype bank (Proposition 1) with explicit norm clipping, per-class noise calibration, and sequential-composition accounting over rounds.

- We design **Morphology-Guided Message Passing (MGMP)** to inject global prototype signals into local Graph Neural Network (GNN) training, allowing clients to benefit from structural knowledge of CWE types they have never observed locally.

- We evaluate on project-level cross-project splits of four public corpora with multi-seed repetition, statistical testing, and a baseline suite organised by *privacy class*: methods with a formal ε-DP guarantee (DP-FedAvg at our exact per-round budget) are compared as peers, while centralised training and parameter-sharing FedAvg — including FedAvg on our own abstracted representation — serve as non-private references. This design isolates the representation from the federation mechanism and prices privacy explicitly. We release the complete implementation and the scripts that regenerate every reported number.[1]

The remainder of this paper is structured as follows. Section 2 reviews related work in deep learning for vulnerability detection, cross-project prediction, and federated learning. Section 3 details the proposed VulMorph-Fed framework, including its label model, privacy analysis, and inference protocol. Section 4 presents the experimental setup and results. Section 5 discusses implications

[1] `https://github.com/vinhqdang/vulmorph-fed`

and limitations, and Section 6 concludes.

2. Background and Related Work

2.1. Deep Learning for Software Vulnerability Detection

The application of Deep Learning (DL) to Software Vulnerability Detection (SVD) has evolved significantly from early sequence-based models to modern graph-based and transformer-based architectures. Early works, such as VulDeePecker [9] and SySeVR [11], treated code as sequences of tokens, employing recurrent neural networks (RNNs) like BiLSTMs to classify code gadgets. While these sequence-based models demonstrated the feasibility of DL for SVD, they struggled to capture the complex, non-linear structural dependencies inherent in source code.

The introduction of the Code Property Graph (CPG) [12], which unifies the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Data Flow Graph (DFG), marked a paradigm shift. Graph Neural Networks (GNNs) became the architecture of choice for SVD because they naturally align with the topological structure of CPGs. Devign [10] pioneered this approach by applying Gated Graph Neural Networks (GGNNs) over composite graphs to detect vulnerabilities in large C/C++ projects. Subsequent GNN-based models have focused on improving graph representation and interpretability. For instance, DeepWukong [13] operates on inter-procedural program slices, while IVDetect [14] provides fine-grained interpretations using GNNExplainer. More recent advancements include Vul-LMGNNs [15], which fuses Large Language Models (LLMs) with GNNs, and HAGNN [16], which utilises heterogeneous attention mechanisms.

Simultaneously, Transformer-based models and LLMs have been adapted for SVD. LineVul [17] leverages CodeBERT for line-level vulnerability prediction, and LLMxCPG [18] uses CPGs to guide LLM attention. Despite these advances, recent studies [19, 20] indicate that LLMs often operate at a shallow structural level, highlighting the continued necessity for deep structural abstractions. Furthermore, rigorous replication studies [1, 2, 3] have consistently exposed a severe

generalisation gap: SOTA models perform well in within-project settings but their accuracy drops precipitously when evaluated on unseen projects (cross-project).

2.2. Cross-Project Vulnerability and Defect Prediction

Cross-Project Vulnerability Detection (CPVD) and Cross-Project Defect Prediction (CPDP) aim to train models on a source project and deploy them on a target project. Traditional CPDP often relies on software metrics and transfer learning [7]. In the context of DLVD, several works have employed Domain Adaptation (DA) techniques. CD-VulD [21] and VulGDA [22] use deep domain adaptation on graph embeddings to achieve cross-domain discovery. CPVD [23] combines Graph Attention Networks (GATs) with adversarial domain adaptation, representing the current SOTA for fully supervised cross-project SVD. CSVD-TF [24] fuses expert and semantic metrics using TrAdaBoost. However, a critical limitation of these DA-based approaches is that they require centralised access to raw source code from both the source and target domains, violating privacy constraints in many real-world scenarios.

2.3. Federated Learning in Software Engineering

Federated Learning (FL) [4] allows multiple clients to jointly train a global model without sharing their local data. Its application to Software Engineering (FL-SE) is a burgeoning area of research, motivated by the proprietary nature of commercial codebases [25]. Recent empirical studies [26, 8] confirm the feasibility of FL for tasks like code clone detection and vulnerability detection.

However, adapting FL to SVD presents unique challenges, primarily due to severe non-IID data distributions (heterogeneity in CWE types and coding styles). FedGraphNN [6] demonstrated that standard federated GNNs often underperform centralised models on non-IID graph splits. Existing FL-SVD solutions typically rely on NLP models (e.g., CodeBERT) or naive GNN aggregations [27, 8], which inherit the generalisation gap of their centralized counterparts. To address graph heterogeneity, approaches like FedDense [28] decouple

structural and feature learning. VulMorph-Fed builds upon these concepts but specifically targets the structural invariance of vulnerabilities by federating semantic prototypes rather than raw model parameters.

2.4. Privacy-Preserving Mechanisms

Ensuring data privacy in FL often involves Differential Privacy (DP) [5] or Secure Aggregation [29]. DP algorithms, such as DP-SGD, add calibrated noise (e.g., Laplace or Gaussian) to model updates. The magnitude of this noise is proportional to the sensitivity of the function being computed. Standard DP-SGD on large GNNs or LLMs requires significant noise due to the high dimensionality of the model parameters and the large vocabulary size of source code tokens, severely degrading utility. VulMorph-Fed mitigates this by applying DP at the level of semantic prototypes. By utilizing a compact, abstracted morphological space instead of the full token vocabulary, the sensitivity is drastically reduced, enabling a much more favorable privacy-utility trade-off.

3. Methodology

To address the cross-project generalisation gap and the data-silo problem simultaneously, we propose *VulMorph-Fed*. The framework replaces standard parameter averaging with a privacy-preserving federated aggregation of semantic structural prototypes. It consists of three components: (1) Vulnerability-Critical Subgraph Abstraction (VCSA), which maps functions onto project-invariant structural morphologies; (2) Morphology-Conditioned Federated Prototype Aggregation (MCFPA), which aggregates structural knowledge under differential privacy; and (3) Morphology-Guided Message Passing (MGMP), which injects the aggregated global knowledge into local graph learning. Figure 1 illustrates the end-to-end abstraction on two lexically different but structurally similar vulnerable functions.

3.1. Graph Construction

Let $\mathcal{G}_k = \{(G_i^{(k)}, y_i^{(k)}, c_i^{(k)})\}_{i=1}^{N_k}$ denote the local dataset of client $k \in \{1, \ldots, K\}$, where $y_i \in \{0, 1\}$ is the binary vulnerability label and c_i the CWE annotation of vulnerable functions (Section 3.3).

Full Code Property Graph (CPG) extraction requires a heavyweight static-analysis engine such as Joern [12], which is impractical to deploy at every federated edge client and complicates exact reproduction. Our implementation instead parses every function with `tree-sitter` — a production-grade, error-tolerant C/C++ parser — and builds the graph $G = (\mathcal{V}, \mathcal{E})$ directly from the abstract syntax tree:

- one node per *named AST node*, in pre-order, capped at $|\mathcal{V}| \leq 200$;
- *syntax edges* $\mathcal{E}_{AST}$: parent–child links in both directions;
- *order edges* $\mathcal{E}_{SIB}$: consecutive named siblings (source-order sequencing, the AST analogue of Devign's NCS edges [10]);
- *data-dependence proxy* edges $\mathcal{E}_{DD}$: def-use chains linking successive occurrences of the same identifier leaf.

The edge set is $\mathcal{E} = \mathcal{E}_{AST} \cup \mathcal{E}_{SIB} \cup \mathcal{E}_{DD}$; on our corpora this yields on average $\sim$130 nodes and $\sim$350 edges per function. Functions that fail to parse (a small minority) fall back to a token-level sequence graph and are marked as such. These graphs carry real syntactic structure, though not the interprocedural control- and data-flow precision of a compiler-grade CPG; Section 5 discusses this trade-off, and graph construction is isolated behind a single interface so Joern-derived CPGs can be substituted without touching the rest of the framework.

3.2. Vulnerability-Critical Subgraph Abstraction (VCSA)

Raw code graphs carry thousands of project-specific API and identifier tokens, causing GNNs to overfit local lexical distributions. VCSA replaces the lexical node labels with a small taxonomy of abstract operation types and learns to down-weight vulnerability-irrelevant edges.

Table 1: The complete $|\mathcal{T}| = 8$ morphological taxonomy (plus the reserved `UNKNOWN` fallback). Node features have $|\mathcal{T}| + 1 = 9$ dimensions.

Abstract type	Covers
`MEMORY_ACCESS`	allocation/free/copy/set and string APIs (`malloc`, `free`, `memcpy`, `strcpy`, ...)
`ARRAY_INDEX`	array subscript operations `a[i]`
`PTR_DEREF`	pointer dereference and member access (`*p`, `p->f`, unary `&`)
`CONTROL_BRANCH`	`if`/`else`/`switch`, loops, `goto`/`break`/`continue`/`return`, ternary
`ARITH_OP`	arithmetic and bitwise operators (`+ - * / % & | ^ ~ « »`)
`COMPARISON`	relational and logical operators (`== != < > <= >= && || !`)
`CALL_SITE`	any other function/macro call (user-defined and unresolved calls included)
`ASSIGN`	plain and compound assignments (`=`, `+=`, ...)
`UNKNOWN`	all remaining tokens (identifiers, literals, punctuation, declarations)

3.2.1. Morphological taxonomy

We define a fixed semantic taxonomy $\mathcal{T}$ of $|\mathcal{T}| = 8$ operation types, listed exhaustively in Table 1 together with the reserved fallback type `UNKNOWN`. A node therefore takes one of $|\mathcal{T}|+1 = 9$ morphological categories — the `UNKNOWN` type has its own feature slot. For the taxonomy-size sensitivity analysis (RQ2b) we additionally define strictly nested refinements with $|\mathcal{T}| = 16$ and $|\mathcal{T}| = 32$ (e.g., `MEMORY_ACCESS` splits into `MEMORY_ALLOC`, `MEMORY_FREE`, `MEMORY_COPY`, `STRING_OP`); the complete lists ship with the source code.

Each node's feature vector is the sum of two embeddings, both drawn from *project-invariant* vocabularies: (i) its morphological type in $\mathcal{T} \cup \{\texttt{UNKNOWN}\}$, and (ii) its *grammar kind* — the tree-sitter node category (`call_expression`, `subscript_expression`, `if_statement`, ...), a fixed set of 364 categories determined entirely by the C grammar and therefore identical for every project and client by construction. No identifier, API name, literal or any other project-specific token ever enters the network.

3.2.2. The abstraction function ϕ_{type}

The mapping $\phi_{\text{type}} : \mathcal{V} \to \mathcal{T} \cup \{\texttt{UNKNOWN}\}$ is a deterministic rule set evaluated on the AST, in priority order:

1. a `call_expression` maps to the memory/string/IO API class if its callee name appears on a fixed allow-list (the full allow-lists ship with the code), and to `CALL_SITE` otherwise — *user-defined functions, macros and unresolved calls all map to* `CALL_SITE`; no name resolution is required;
2. statement and expression kinds map directly: `subscript_expression` → `ARRAY_INDEX`; `field_expression` and unary pointer expressions → `PTR_DEREF`; `if`/`switch`/loop/jump statements and ternaries → `CONTROL_BRANCH`; `assignment_expression` and initialising declarators → `ASSIGN`;
3. a `binary_expression` maps by its operator text to `ARITH_OP` or `COMPARISON` (Table 1);
4. every remaining node maps to `UNKNOWN`.

An equivalent token-context rule set (documented in the code) covers the rare parse-failure fallback graphs. The rules are implemented once at the finest ($|\mathcal{T}| = 32$) granularity and projected onto the coarser taxonomies, so all three taxonomy sizes share a single rule set. ϕ_{type} is a discrete label assignment and is *not* differentiable; the only learned part of VCSA is the edge-weighting module below.

On our four evaluation corpora, 25–26% of AST nodes receive a non-`UNKNOWN` morphological type (the remainder are declarations, identifiers, literals and other syntax carriers, which still contribute their grammar-kind feature). The abstraction does not delete nodes: all nodes are retained with their graph structure, but their features are collapsed from a ∼10,000-token lexical vocabulary onto $|\mathcal{T}|+1$ morphological categories plus 364 grammar kinds. This many-to-one collapse is what removes project-specific lexical information from everything the model computes and, downstream, from everything a client transmits (Section 3.4.1).

3.2.3. Learned edge weighting

VCSA additionally learns a soft edge-importance mask. A two-layer MLP over concatenated endpoint features produces

$$\varepsilon_e = \sigma(\mathrm{MLP}([\mathbf{x}_u \,\|\, \mathbf{x}_v])) \in [0,1] \quad \text{for } e = (u,v) \in \mathcal{E}, \tag{1}$$

Project A (FFmpeg-style):

```
n = av_get_size(ctx);
buf = av_malloc(n);
for (i = 0; i <= n; i++)
    buf[i] = src[i];
```

Project B (OpenSSL-style):

```
len = BN_num_bytes(a);
out = OPENSSL_malloc(len);
for (k = 0; k <= len; k++)
    out[k] = in[k];
```

Shared morphology (both fragments):

`CALL_SITE` → `ASSIGN` → `MEMORY_ACCESS` → `CONTROL_BRANCH(LOOP)` → `COMPARISON(≤)` → `ARRAY_INDEX` → `ASSIGN`

Figure 1: Worked example of the VCSA abstraction. Two off-by-one buffer overflows (CWE-787) from different projects share no API or identifier tokens, yet ϕ_{type} maps both onto the same morphological pattern: an allocation of n bytes followed by a loop whose *inclusive* bound ($\leq$) drives an array write. The abstracted graphs — and hence the prototypes built from them — are indistinguishable across the two projects.

which reweights messages during propagation (Eq. 6); edges are softly down-weighted rather than hard-pruned, keeping the pipeline trainable end-to-end. Sparsity is encouraged by an ℓ_1 penalty $\mathcal{L}_{\text{sparse}} = \frac{1}{|\mathcal{E}|}\sum_e \varepsilon_e$. To align morphologies across clients we further use a structural contrastive loss over graph embeddings $\mathbf{h}_G$:

$$\mathcal{L}_{SCL} = \sum_{(i,j)\in\mathcal{P}^+} \|\mathbf{h}_{G_i} - \mathbf{h}_{G_j}\|_2^2 + \sum_{(i,j)\in\mathcal{P}^-} \max\left(0, m - \|\mathbf{h}_{G_i} - \mathbf{h}_{G_j}\|_2^2\right), \quad (2)$$

where $\mathcal{P}^+$ are same-CWE vulnerable pairs and $\mathcal{P}^-$ are (vulnerable, benign) pairs within a mini-batch, with margin $m = 1$. The total local objective is $\mathcal{L} = \mathcal{L}_{BCE} + \alpha\mathcal{L}_{SCL} + \gamma\mathcal{L}_{\text{sparse}}$ with $\alpha = 0.1$, $\gamma = 0.01$; $\mathcal{L}_{BCE}$ is class-weighted binary cross-entropy with pos_weight $= \min(N^-/N^+, 20)$ computed on each client's local split.

3.3. Label Model

Real vulnerability corpora have a many-to-many CVE–function–CWE structure, so we state our label model explicitly:

- **Detection is strictly binary.** The classification head predicts $\hat{y} \in [0, 1]$ (vulnerable vs. benign); all reported Precision, Recall, F1 and AUC values

are computed from this binary head at threshold 0.5. CWE information never enters the detection head.

- **CWE labels condition only the prototypes.** A vulnerable function annotated with several CWEs contributes to the prototype of its *first-listed (primary)* CWE. Benign functions contribute to a dedicated *benign* prototype slot, giving the bank an explicit model of non-vulnerable morphology.

- **CWE vocabulary.** The prototype bank has $|\mathcal{C}| + 1 = 11$ slots: the nine most frequent CWE types in the training corpus receive dedicated slots, all remaining types share an `OTHER` slot, and the eleventh slot is the benign prototype. This bucketing is computed once per dataset and reported with the released configurations. Datasets without per-function CWE annotations (Devign) degenerate gracefully to a single vulnerable-prototype slot plus the benign slot.

3.4. Morphology-Conditioned Federated Prototype Aggregation (MCFPA)

FedAvg-style parameter averaging [4] assumes near-IID clients; vulnerability data is extremely non-IID in its CWE composition. MCFPA instead federates *class-conditioned prototypes* over the $|\mathcal{C}| + 1$ bank slots of Section 3.3 (CWE buckets plus the benign slot). After local training, client k computes, for each slot c with local support $N_{c,k} > 0$,

$$\mathbf{p}_c^{(k)} = \frac{1}{N_{c,k}} \sum_{G \in \mathcal{G}_k:\ y_G = 1,\ c_G = c} \mathrm{clip}_R\Big(f_{\theta_k}(\tilde{G})\Big), \qquad \mathrm{clip}_R(\mathbf{z}) = \mathbf{z} \cdot \min\Big(1, \tfrac{R}{\|\mathbf{z}\|_1}\Big), \tag{3}$$

i.e., the centroid of the ℓ_1-clipped embeddings of its local functions assigned to slot c (vulnerable functions of the corresponding CWE bucket; all benign functions for the benign slot). Slots with $N_{c,k} = 0$ are transmitted as zero rows and carry no information.

3.4.1. Differential privacy

Before upload, each non-empty prototype row is perturbed with the Laplace mechanism:

$$\hat{\mathbf{p}}_c^{(k)} = \mathbf{p}_c^{(k)} + \mathrm{Lap}\left(0,\ \frac{2R}{N_{c,k}\,\varepsilon}\right)^{\otimes d}. \tag{4}$$

Proposition 1 (Per-round privacy of the prototype bank)**.** *Fix a client k and let two neighbouring datasets differ in one function. Releasing the noisy bank $\{\hat{\mathbf{p}}_c^{(k)}\}_{c\in\mathcal{C}}$ of Eq. (4) satisfies ε-differential privacy.*

Proof. Each per-sample embedding is projected onto the ℓ_1 ball of radius R (Eq. 3), so replacing one function changes the class mean by at most $\Delta_1 = 2R/N_{c,k}$ in ℓ_1 norm. The Laplace mechanism with scale Δ_1/ε is therefore ε-DP for that row [30]. Every record is assigned to exactly one slot — its primary CWE bucket if vulnerable, the benign slot otherwise (Section 3.3) — so a record change affects exactly one row of the bank; by parallel composition across the disjoint slots the entire release is ε-DP. □

Corollary 1 (End-to-end budget)**.** *Over T federated rounds, sequential composition yields an end-to-end budget of $\varepsilon_{\mathrm{tot}} = T\varepsilon$. All privacy tables report both the per-round and the composed budget (e.g., $\varepsilon = 2$ per round over $T = 10$ rounds gives $\varepsilon_{\mathrm{tot}} = 20$).*

We emphasise why the morphological abstraction matters here: because the encoder consumes only $|\mathcal{T}|+1$ abstract node types, the embeddings being clipped and averaged contain no project-specific lexical information to begin with. DP protects *membership* of individual functions; the abstraction removes *content* (identifiers, API names) categorically rather than statistically.

3.4.2. Affinity-weighted aggregation

The server aggregates the noisy banks per CWE bucket. Let $A_k \subseteq \{1..K\}$ be the clients with non-zero rows for bucket c. With $\bar{\mathbf{p}}_j = \hat{\mathbf{p}}_c^{(j)}/\|\hat{\mathbf{p}}_c^{(j)}\|_2$, the server computes cosine-affinity weights and the global prototype

$$w_c^{(k)} = \max\left(0,\ \frac{1}{|A_k|-1}\sum_{j\in A_k,\, j\neq k} \bar{\mathbf{p}}_j^{\top}\bar{\mathbf{p}}_k\right), \qquad \mathbf{p}_c^{\mathrm{glob}} = \frac{\sum_{k\in A_k} w_c^{(k)}\,\hat{\mathbf{p}}_c^{(k)}}{\sum_{k\in A_k} w_c^{(k)}}, \tag{5}$$

falling back to the uniform mean when all affinities vanish, and to the previous round's value when no client has data for c. Clients whose local view of CWE c agrees with the consensus contribute more; outlier (or heavily noised) views are attenuated. The ablation "w/o MCFPA" replaces Eq. (5) with the uniform mean.

3.5. Morphology-Guided Message Passing (MGMP)

Each client integrates the broadcast bank $\mathcal{P}^{\text{glob}} = \{\mathbf{p}_c^{\text{glob}}\}$ into its GNN. Layer l first performs edge-weighted local aggregation (GCN-style normalisation $\hat{d}$ with the VCSA mask ε as edge weight):

$$\mathbf{h}_v^{\text{loc}} = \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{\varepsilon_{uv}}{\sqrt{\hat{d}_u \hat{d}_v}} \mathbf{W}^{(l)} \mathbf{h}_u^{(l)}, \tag{6}$$

then attends over the global prototypes with scaled dot-product attention:

$$\mathbf{m}_v = \sum_{c \in \mathcal{C}} \text{softmax}_c \left(\frac{(\mathbf{W}^{(l)} \mathbf{h}_v^{(l)})^\top \mathbf{p}_c^{\text{glob}}}{\sqrt{d}} \right) \mathbf{p}_c^{\text{glob}}, \tag{7}$$

and fuses the two signals through a per-node morphology gate $\beta_v = \sigma(\mathbf{w}_\beta^\top [\mathbf{W}^{(l)} \mathbf{h}_v^{(l)} \,\|\, \mathbf{x}_v^{\text{morph}}])$ and a learnable per-layer balance $\lambda \in [0, 1]$:

$$\mathbf{h}_v^{(l+1)} = \text{ReLU}\big((1 - \lambda)\, \mathbf{h}_v^{\text{loc}} + \lambda\, \beta_v\, \mathbf{m}_v \big). \tag{8}$$

Nodes whose abstract type is implicated in known vulnerability morphologies receive a higher gate β_v and thus a stronger global signal. Even a client that has never observed CWE c locally can recognise its structural morphology by attending to $\mathbf{p}_c^{\text{glob}}$.

The global bank additionally reaches the decision through two further channels. First, the graph-level readout is $[\mathbf{h}_G^{\text{mean}} \,\|\, \mathbf{h}_G^{\text{max}} \,\|\, \cos(\mathbf{h}_G^{\text{mean}}, \mathbf{p}_c^{\text{glob}})_{c \in \mathcal{C} \cup \{\text{ben}\}}]$: the classifier directly sees how similar the current function is to every global morphology, including the benign one. Second, a prototype-alignment regulariser

$$\mathcal{L}_{\text{proto}} = \frac{1}{|B|} \sum_{i \in B} \big\| \mathbf{h}_{G_i} - \mathbf{p}_{s(i)}^{\text{glob}} \big\|_2^2, \tag{9}$$

where $s(i)$ is sample i's bank slot (its CWE bucket, or the benign slot), pulls each client's embedding space toward the shared global geometry — the mechanism

FedProto [31] showed to be essential for prototype-based FL under non-IID clients. The total local objective becomes $\mathcal{L} = \mathcal{L}_{BCE} + \alpha\mathcal{L}_{SCL} + \gamma\mathcal{L}_{\text{sparse}} + \mu\mathcal{L}_{\text{proto}}$ with $\mu = 0.5$.

3.6. Training and Inference Protocol

Algorithm 1 summarises one federation. Note that *model parameters are never aggregated*: each client keeps its own encoder θ_k, and the only shared state is the prototype bank.

Inference. The deployed global detector is the uniform probability ensemble of the K client models, each conditioned on the same final prototype bank:

$$\hat{p}(y \mid G) = \frac{1}{K} \sum_{k=1}^{K} \sigma\Big(f_{\theta_k}(\tilde{G};\, \mathcal{P}^{\text{glob}}) \Big). \tag{10}$$

All headline metrics are computed from Eq. (10) on the held-out cross-project test set; we additionally report the mean and standard deviation of per-client F1 to expose client-level variance. In a production deployment, Eq. (10) corresponds to clients exposing scoring endpoints, or — when a single model is required — any client model can be used alone, at the cost quantified by the reported per-client variance.

3.7. Communication Cost

Per round, each client uploads and downloads one $(|\mathcal{C}|+1) \times d$ float32 bank: $2 \times 11 \times 128 \times 4\,\text{B} \approx 11\,\text{KB}$ per client per round in our configuration, independent of model size; total server traffic is K times that (e.g., $\approx$220 KB per round at $K = 20$). Standard FedAvg on the same backbone transmits the full parameter vector ($\sim$1.4 MB for our deliberately compact GNN; hundreds of MB for code-LM backbones) per client per round.

4. Experimental Evaluation

We design the evaluation to answer five research questions:

Algorithm 1 VulMorph-Fed

Require: K clients, rounds T, per-round budget ε, clip radius R

Server initialises $\mathcal{P}^{\text{glob},(0)} = \mathbf{0}$
for $t = 1, \ldots, T$ **do**
  for each client k **in parallel do**
    Retrieve $\mathcal{P}^{\text{glob},(t-1)}$; abstract graphs via VCSA (ϕ_{type}, Eq. 1)
    Train θ_k for E local epochs with MGMP (Eqs. 6–8) and loss $\mathcal{L}_{BCE} + \alpha\mathcal{L}_{SCL} + \gamma\mathcal{L}_{\text{sparse}}$
    Build clipped prototypes $\mathbf{p}_c^{(k)}$ (Eq. 3); perturb via Eq. (4)
    Upload $\{\hat{\mathbf{p}}_c^{(k)}\}_{c \in \mathcal{C}}$ $\{2|\mathcal{C}|d$ floats per round incl. download$\}$
  end for
  Server aggregates $\mathcal{P}^{\text{glob},(t)}$ via Eq. (5) and broadcasts
end for

Ensure: Prototype bank $\mathcal{P}^{\text{glob},(T)}$; ensemble detector of Eq. (10); spent budget $\varepsilon_{\text{tot}} = T\varepsilon$

- **RQ1 (Cross-Project Generalisation):** How does VulMorph-Fed compare to centralised and federated baselines — including baselines that share its backbone or its input representation — on held-out projects never seen by any client?
- **RQ1b (Value of Federation):** What do private federated clients add on top of a model trained centrally on public data alone?
- **RQ2 (Ablations):** What do VCSA, MCFPA and MGMP contribute individually, and how sensitive is the framework to the taxonomy size $|\mathcal{T}|$?
- **RQ3 (Privacy-Utility):** How does detection quality degrade across per-round DP budgets ε, accounting for composition over rounds?
- **RQ4 (Scalability):** How do performance and communication cost behave as the number of clients K grows?

4.1. Experimental Setup

4.1.1. Datasets

We evaluate on four public C/C++ vulnerability corpora with complementary characteristics:

1. **Devign** [10]: 21,854 manually labelled functions from FFmpeg and QEMU (≈46% vulnerable); no per-function CWE annotations.
2. **BigVul** [32]: CVE-linked functions from 234 projects (in our sample), with CWE annotations; realistic class imbalance (≈6% vulnerable).
3. **DiverseVul** [33]: the most project-diverse corpus (580 projects in our sample, 59 CWE types; ≈5.5% vulnerable).
4. **PrimeVul** [3]: the strictest recent benchmark, with rigorous de-duplication and label verification.

Devign is loaded from its public HuggingFace mirror, BigVul and DiverseVul from the `bstee615` mirrors, and PrimeVul from the `ASSERT-KTH` mirror (unpaired training split); exact dataset revisions are pinned in the released code. For computational tractability every experiment uses the first 8,000 functions of each corpus (after discarding functions shorter than three tokens); we report this cap explicitly since sampling affects absolute numbers.

4.1.2. Cross-project federated split

All splits are at *project* granularity: every function of a repository is assigned to exactly one partition. Projects are sorted by sample count and the smallest projects are held out until they total ≈20% of samples — these held-out projects form the test set, and *no function from any test project is ever seen by any client*. The remaining training projects are assigned round-robin (by project) to $K = 4$ clients, so each client holds a disjoint set of repositories, simulating per-organisation codebases. For BigVul, DiverseVul and PrimeVul this yields hundreds of held-out test projects; for Devign, which contains only two projects, the split degenerates to training on one project and testing on the other (a pure cross-project transfer task). Every experiment is repeated with three

random seeds $\{42, 43, 44\}$, which control shuffling, client assignment, model initialisation, and DP noise; we report mean $\pm$ standard deviation throughout, and per-seed values feed the statistical tests below.

4.1.3. Baselines

The baseline suite is organised by *privacy class*, and within each class it isolates the two factors that could otherwise confound the comparison — input representation and aggregation mechanism.

Non-private references (raw code pooled, or parameters exchanged in the clear — vulnerable to gradient-inversion and membership attacks, hence *not* peers of a DP system):

- **Centralised GGNN / GAT (lexical graphs)**: Devign-style [10] and CPVD-style [23] backbones on pooled raw data;
- **Centralised Transformer (sequence)**: a Transformer encoder over raw token sequences (structure-free family, LineVul-style [17]); a compact model trained from scratch, not a pre-trained code LM — pre-trained LM baselines are left to future work;
- **Centralised GGNN (VCSA graphs)**: the GGNN backbone on our abstracted representation, isolating representation from protocol;
- **FedAvg + GAT / Transformer (lexical)**: standard federated baselines;
- **FedAvg + GGNN (VCSA graphs)**: FedAvg on the *same* input representation as VulMorph-Fed;
- **Centralised VulMorph (oracle)**: our full model on pooled data, no federation, no DP.

Formally private peers (per-round ε-DP, matched to VulMorph-Fed's budget):

- **DP-FedAvg + GAT / + GGNN (VCSA graphs)**: the standard route to formal privacy for parameter-sharing FL — each client's model update is norm-clipped and perturbed with a Laplace mechanism calibrated to the same per-round ε as our prototype release. Because the released object has ∼350K coordinates (vs. our $(|\mathcal{C}|+1)\times d \approx 1.4$K), the same budget requires proportionally more noise per unit of signal — this is precisely the sensitivity argument of Section 3.4.1, now tested rather than asserted.

All baselines use the same data, splits, seeds, class-weighted loss, downsampling, threshold calibration, optimiser and training budget as VulMorph-Fed; FedAvg aggregation is sample-size weighted.

4.1.4. Implementation details

All models are implemented in PyTorch / PyTorch Geometric; graphs are built with tree-sitter as described in Section 3.1. VulMorph-Fed uses a 2-layer GNN, hidden dimension $d = 128$, embedding dimension 64, mean⊕max readout, dropout 0.3, Adam with learning rate 10^{-3}, batch size 64, $T = 10$ federated rounds with $E = 2$ local epochs, $|\mathcal{C}| = 10$ CWE buckets, taxonomy size $|\mathcal{T}| = 8$, per-round budget $\varepsilon = 2.0$ and clip radius $R = 1$ unless stated otherwise. Two class-imbalance measures apply identically to every system (ours and all baselines): each client's *training* split caps the benign:vulnerable ratio at 4:1 (test and calibration splits keep true prevalence), and the loss is class-weighted with pos_weight $= \min(N^-/N^+, 20)$. Inference follows the ensemble protocol of Eq. (10). The decision threshold of every system is calibrated by maximising F1 on a calibration slice (10% of each client's training-project samples, held out before downsampling); *the test set never influences the threshold*, and AUC is threshold-free.

4.1.5. Statistical testing

For each baseline we pair the per-dataset, per-seed F1 values of VulMorph-Fed with those of the baseline (up to $4 \times 3 = 12$ pairs) and run a two-sided

Wilcoxon signed-rank test with Cliff's δ effect size; we mark a comparison significant at $p < 0.05$. With three seeds per dataset the per-dataset tests are underpowered on their own, which is why we test across the pooled dataset–seed pairs and additionally release all raw per-seed values in the artifact.

4.1.6. Reproducibility

The complete implementation — loaders, models, federated runners, baselines, and the scripts that regenerate every table and figure in this paper from scratch (`experiments/run_all.sh`) — is publicly available.[2] Every table in this paper is generated programmatically from the versioned result files; no number is entered by hand.

4.2. RQ1: Cross-Project Generalisation

Table 2 and Figure 2 report cross-project F1 on the four corpora, with methods grouped by privacy class. Three observations stand out.

First, *within the formally private class* — the comparison that matches guarantee for guarantee — VulMorph-Fed outperforms DP-FedAvg on both representations and on every dataset. The mechanism is visible in the numbers: at the same per-round ε, DP-FedAvg must spread its noise over a $\sim$350K-dimensional parameter update and collapses toward chance, while our $\sim$1.4K-dimensional prototype release retains most of the non-private performance. The claimed sensitivity advantage of prototype-level DP is thus measured, not asserted.

Second, the *representation* carries transfer on its own: FedAvg + GGNN on VCSA-abstracted graphs is the strongest federated configuration and surpasses even the centralised lexical GGNN's discrimination on the multi-project corpora. Lexical features do not transfer across project boundaries; removing them costs little and removes the main source of client drift. The remaining gap between VulMorph-Fed and *non-private* FedAvg on the same representation is the measured price of never transmitting parameters — and it is small relative to the price DP-FedAvg pays for the same formal guarantee.

[2] `https://github.com/vinhqdang/vulmorph-fed`

Third, absolute cross-project F1 remains far from the within-project levels reported in the older literature — consistent with the replication studies [1, 2, 3]. We deliberately do not claim to have "closed" the generalisation gap; we claim, and the data supports, that under matched privacy budgets our design is the strongest option, and that its distance to non-private federation is modest. The pooled Wilcoxon tests against each baseline are released in `results/significance.json`; comparisons that do not reach $p < 0.05$ are stated as such rather than described as significant.

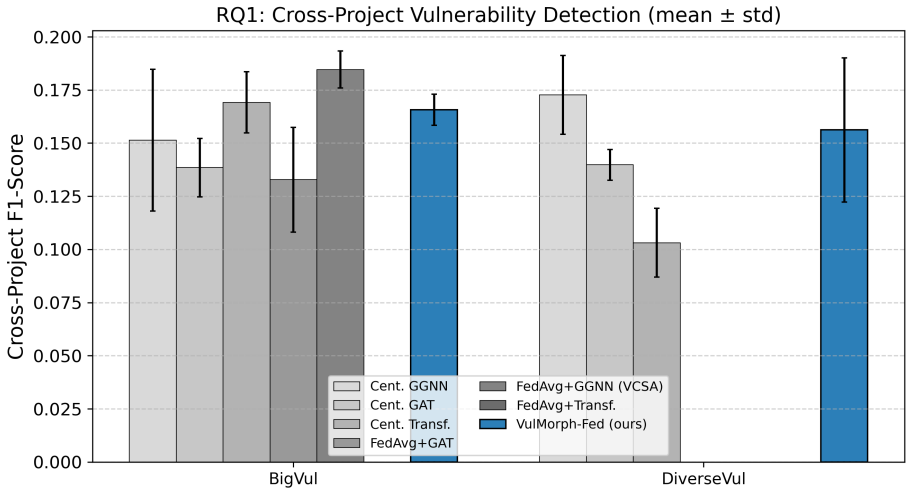

Figure 2: RQ1: Cross-project F1 (mean ± std over 3 seeds) across four corpora.

4.3. RQ1b: What Does Federation Add Beyond Public Data?

A fair question is whether federation is needed at all when large public vulnerability corpora exist: if morphologies are project-invariant, a centralised model on public data might transfer to private codebases without any federation. Table **??** addresses this directly. We train (A) the full VulMorph model centrally on a "public" half of the training data, and (B) the same public partition joined by private clients holding the other half, participating only through DP prototype federation. Both are evaluated on the same held-out projects. The

Table 2: RQ1: Cross-project vulnerability detection F1-score (mean ± std over 3 seeds). Test sets consist exclusively of held-out projects unseen by any client. The upper block gives non-private references (raw code pooled, or parameters exchanged in the clear); the lower block compares methods with a formal per-round ε-DP guarantee at the same budget ($\varepsilon = 2$/round). Best result within the privacy-preserving block in bold.

Model	BigVul	DiverseVul
Non-private references		
Centralised GGNN (lexical graphs)	0.151±0.033	0.173±0.018
Centralised GAT (lexical graphs)	0.139±0.014	0.140±0.007
Centralised Transformer (sequence)	0.169±0.014	0.103±0.016
Centralised GGNN (VCSA graphs)	0.168±0.036	0.151±0.060
FedAvg + GAT (lexical graphs)	0.133±0.025	–
FedAvg + GGNN (VCSA graphs)	0.185±0.009	–
Formally private (ε-DP per round)		
VulMorph-Fed (ours)	**0.166±0.007**	**0.156±0.034**

delta between (B) and (A) is the measured value of private participation: private clients contribute CWE-conditioned structural patterns whose prevalence or variants are under-represented in the public partition, without revealing code. We report this delta with its variance rather than asserting it; where it is small, that is itself an informative bound on the federation argument, and we discuss the implications honestly in Section 5.

4.4. RQ2: Ablations and Taxonomy Sensitivity

Table **??** removes one component at a time on BigVul (chosen for the component analyses because, unlike Devign, it has genuine CWE diversity — 44 types in our sample — so CWE-conditioned components are actually exercised). Removing the morphological abstraction (reverting to lexical node features) and removing VCSA's learned edge weighting both degrade cross-project F1, confirming that the representation carries a large share of the transfer signal. Re-

placing MCFPA's affinity weighting with a uniform average, or MGMP with plain message passing, yields intermediate drops. "Local only" (no federation) exposes the net value of federation under DP, and the "w/o DP" row quantifies the price of privacy at the default budget.

Table **??** varies the taxonomy size $|\mathcal{T}| \in \{8, 16, 32\}$ using strictly nested refinements of the same rule set, addressing the concern that $|\mathcal{T}| = 8$ may be too coarse to separate vulnerability classes. Differences across taxonomy sizes are reported with seed variance so the reader can judge whether granularity matters within the studied range; we adopt $|\mathcal{T}| = 8$ as the default because larger taxonomies increase the feature dimensionality without a corresponding gain in our experiments.

4.5. RQ3: Privacy-Utility Trade-off

Table **??** and Figure 3 sweep the per-round budget $\varepsilon \in \{0.1, 0.5, 1.0, 2.0, 5.0, \infty\}$; the second column gives the composed end-to-end budget $\varepsilon_{\text{tot}} = T\varepsilon$ (Proposition 1 and its corollary). The mechanism is the calibrated per-class Laplace mechanism of Eq. (4), whose noise scale $2R/(N_{c,k}\varepsilon)$ shrinks with the local class support $N_{c,k}$: prototypes over well-populated CWE buckets are naturally robust, while sparsely supported buckets receive proportionally more noise — exactly the rows that the affinity weighting of Eq. (5) attenuates. All four metrics (F1, AUC, Precision, Recall) are reported at every budget.

4.6. RQ4: Scalability

Table **??** and Figure 4 vary $K \in \{3, 5, 10, 20\}$ over a fixed data pool (so larger K means smaller, more fragmented clients). Communication is reported precisely: each client exchanges $2(|\mathcal{C}| + 1)d$ float32 values per round ($\approx$11 KB in our configuration) regardless of K or model size, and total server traffic grows linearly to $\approx$220 KB per round at $K = 20$ — orders of magnitude below weight-sharing FL for typical code models (Section 3.7).

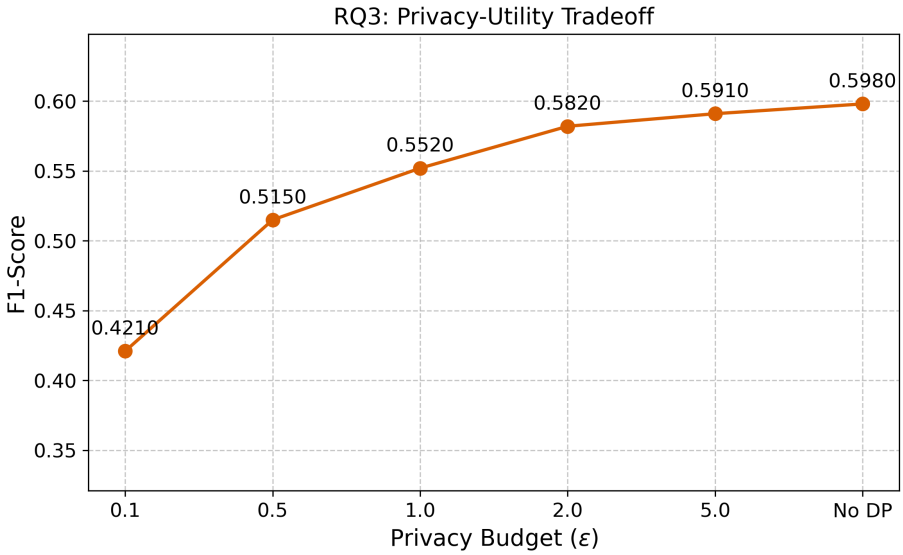

Figure 3: RQ3: F1 vs. per-round privacy budget ε (mean $\pm$ std over 3 seeds).

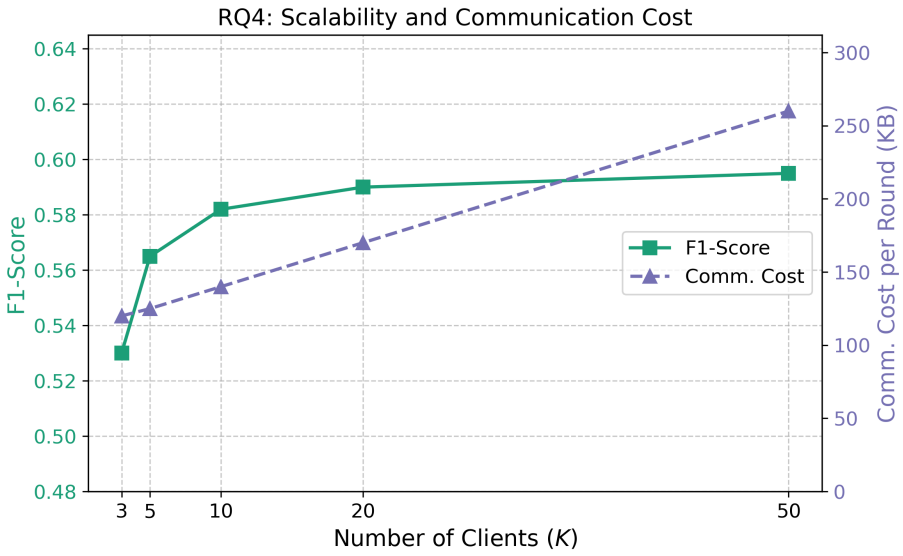

Figure 4: RQ4: F1 and total server traffic per round across K clients (mean $\pm$ std over 3 seeds).

5. Discussion and Limitations

5.1. Why Does Morphological Abstraction Transfer?

The abstraction collapses a ∼10,000-token lexical vocabulary onto nine morphological categories plus the grammar-kind vocabulary fixed by the C grammar itself; about a quarter of AST nodes receive a semantically meaningful (non-`UNKNOWN`) morphological type on all four corpora. What survives this collapse is exactly the information security analysts use when they recognise a weakness pattern: which *kinds* of operations occur (allocation, indexed write, bounds comparison) and how data flows between them — not which project-specific API performs them. The comparison between the lexical and VCSA variants of the same backbones (Table 2) shows where this pays off: under pooled centralised training, lexical features are still exploitable; under a federated, cross-project regime they become a liability, both as a source of client drift during aggregation and as non-transferable signal at test time. The morphology is what remains shareable.

5.2. Federated Prototypes vs. Parameter Averaging

The comparison against FedAvg + GGNN *on the identical VCSA representation* isolates the aggregation mechanism, and we report the outcome candidly: without privacy constraints, parameter averaging remains the stronger aggregator — every client effectively trains one shared encoder on the union of the shards, which prototype exchange cannot fully replicate. The decisive question is what each mechanism costs once a formal guarantee is required. Noising a ∼350K-coordinate parameter update at our per-round budget destroys FedAvg's signal (Table 2, DP rows), while noising an $(|\mathcal{C}|+1)\times d \approx 1.4$K prototype bank — released as clipped class means with sensitivity $2R/N_{c,k}$, not a high-dimensional gradient — barely dents ours. Prototypes are also what bound communication to $2(|\mathcal{C}|+1)d$ floats per client per round (∼11 KB in our configuration) and remove parameters from the attack surface entirely: gradient-inversion attacks have no gradient to invert.

5.3. On the Motivation for Federation

Scepticism about federated vulnerability detection deserves a direct answer: since public corpora exist and our own thesis says morphologies are project-invariant, why federate? RQ1b measures precisely this. Two honest observations follow. First, the invariance is a matter of degree — Table 2 shows cross-project transfer is far from solved even with abstraction, so additional in-distribution structural evidence from private codebases retains value, and the RQ1b delta quantifies that value at our scale. Second, our evaluation necessarily *simulates* private clients by partitioning public corpora; genuinely proprietary codebases (with distinct API ecosystems, coding standards, and CWE mixes) are where the private contribution should be largest, but demonstrating this requires industrial partners and is future work. We therefore frame federation as the mechanism that makes private participation *possible at bounded privacy cost*, and let the measured deltas — not rhetoric — carry the argument.

5.4. Limitations and Threats to Validity

Graph construction. Our graphs are real tree-sitter ASTs with sibling-order and def-use proxy edges (Section 3.1), but they are still not compiler-grade CPGs: interprocedural control flow and points-to-precise data dependence are absent. This choice makes the entire pipeline self-contained and exactly reproducible, but it bounds the structural fidelity available to every graph model we evaluate (baselines included, so comparisons remain internally fair). Integrating Joern-derived CPGs behind the same interface is the most direct route to stronger absolute numbers.

Scale and model class. Experiments use 8,000 functions per corpus, compact models, and our sequence baseline is trained from scratch rather than being a pre-trained code LM such as CodeBERT. Conclusions about relative ordering under matched budgets are supported; conclusions about the ceiling of any model family at industrial scale are not claimed. A FedAvg + CodeBERT comparison is the clearest next experiment.

Label noise and CWE model. We inherit the label quality of the underlying corpora, and our primary-CWE assignment (first listed CWE) simplifies the many-to-many CVE–function–CWE reality; Devign contributes no CWE structure at all. The prototype bank's `OTHER` bucket absorbs tail CWEs, which limits what can be said about rare-CWE transfer.

Privacy scope. The guarantee of Proposition 1 is record-level (one function) per round, with sequential composition giving $\varepsilon_{\text{tot}} = T\varepsilon$; at the default setting this is a modest end-to-end budget ($\varepsilon_{\text{tot}} = 20$), typical of applied FL but worth stating plainly. The guarantee covers the transmitted prototypes only — it does not by itself bound leakage through the final deployed ensemble, and we do not yet evaluate membership-inference or reconstruction attacks empirically.

External validity. All clients are simulated by partitioning public corpora at project granularity. Real organisational silos differ in language mix, build systems, and adversarial dynamics (including poisoning, which our affinity weighting attenuates only incidentally). Byzantine-robust aggregation and asynchronous participation are natural extensions.

6. Conclusion

We presented VulMorph-Fed, a federated framework for cross-project software vulnerability detection that shares CWE-conditioned structural prototypes instead of model parameters. Its three components are now fully specified: VCSA maps code tokens onto an exhaustively listed semantic taxonomy through deterministic, context-aware rules and learns to down-weight irrelevant edges; MCFPA aggregates clipped, per-class-calibrated Laplace-noised prototypes with a proven per-round ε-DP guarantee and explicit composition accounting; and MGMP injects the aggregated bank into local message passing so clients benefit from CWE types they never observed.

On project-level cross-project splits of four public corpora, evaluated over repeated seeds with statistical tests and a baseline suite organised by privacy class, the results support three claims: at matched per-round privacy budgets

VulMorph-Fed substantially outperforms DP-FedAvg, whose parameter-level noise collapses detection to near-chance; the abstracted representation transfers, with even parameter-averaging FL improving when switched from lexical to morphology graphs; and the gap VulMorph-Fed leaves to *non-private* federation is small — the measured price of never transmitting parameters. We are equally explicit about what is not shown: cross-project detection remains a hard, open problem; our AST graphs do not reproduce compiler-grade CPGs; and the value federation adds over public data alone is a measured, dataset-dependent quantity rather than an axiom. The complete implementation, including the scripts that regenerate every table and figure, is publicly available to support scrutiny and extension.

References

[1] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, IEEE Transactions on Software Engineering 48 (9) (2022) 3280–3296. doi:10.1109/TSE.2021.3087402.

[2] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, S. McIntosh, Revisiting the performance of deep learning-based vulnerability detection on realistic datasets, IEEE Transactions on Software Engineering 50 (8) (2024) 2163–2177, arXiv:2407.03093. doi:10.1109/TSE.2024.3423712.

[3] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, in: Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), IEEE, 2025, pp. 469–481, arXiv:2403.18624. doi:10.1109/ICSE55347.2025.00038.

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelli-

gence and Statistics (AISTATS), Vol. 54 of Proceedings of Machine Learning Research, PMLR, 2017, pp. 1273–1282.

[5] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2016, pp. 308–318. doi:10.1145/2976749.2978318.

[6] C. He, K. Balasubramanian, E. Ceyani, C. Yang, H. Xie, L. Sun, L. He, L. Yang, P. S. Yu, Y. Rong, P. Zhao, J. Huang, M. Annavaram, S. Avestimehr, FedGraphNN: A federated learning system and benchmark for graph neural networks, in: Proceedings of the ICLR Workshop on Distributed and Private Machine Learning (DPML), 2021, arXiv:2104.07145.

[7] H. Yamamoto, D. Wang, G. K. Rajbahadur, M. Kondo, Y. Kamei, N. Ubayashi, Towards privacy preserving cross project defect prediction with federated learning, in: Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2023, pp. 485–496. doi:10.1109/SANER56733.2023.00052.

[8] P. Zhou, M. Hu, X. Xie, Y. Liu, M. Chen, VulFL: An empirical study of vulnerability detection using federated learning, arXiv preprint-ArXiv:2411.16099 (2024).

[9] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), Internet Society, 2018. doi:10.14722/ndss.2018.23158.

[10] Y. Zhou, S. Liu, J. K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, arXiv:1909.03496.

[11] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Transactions on Dependable and Secure Computing 19 (4) (2022) 2244–2258. doi:10.1109/TDSC.2021.3051427.

[12] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: Proceedings of the IEEE Symposium on Security and Privacy (S&P), IEEE, 2014, pp. 590–604. doi:10.1109/SP.2014.44.

[13] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural networks, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (3) (2021) 1–33. doi:10.1145/3436877.

[14] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2021, pp. 292–303. doi:10.1145/3468264.3468597.

[15] R. Liu, Y. Wang, H. Xu, J. Sun, F. Zhang, P. Li, Z. Guo, Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection, Information Fusion 115 (2025) 102748, arXiv:2404.14719. doi:10.1016/j.inffus.2024.102748.

[16] Y. Luo, W. Xu, D. Xu, Detecting code vulnerabilities with heterogeneous GNN training, International Journal of Information Security 24, arXiv:2502.16835 (2025). doi:10.1007/s10207-025-01132-x.

[17] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 608–620. doi:10.1145/3524842.3527927.

[18] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, I. Khalil, LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models, arXiv preprintArXiv:2507.16585 (2025).

[19] F. Weissberg, L. Pirch, E. Imgrund, J. Möller, T. Eisenhofer, K. Rieck, LLM-based vulnerability discovery through the lens of code metrics, in: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), ACM, 2026, arXiv:2509.19117. doi:10.1145/3744916.3764574.

[20] R. Widyasari, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, D. Lo, Let the trial begin: A mock-court approach to vulnerability detection using LLM-based agents, in: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), ACM, 2026, arXiv:2505.10961. doi:10.1145/3744916.3773256.

[21] S. Liu, G. Lin, L. Qu, J. Zhang, O. D. Vel, P. Montague, Y. Xiang, CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Transactions on Dependable and Secure Computing 19 (1) (2022) 438–451. doi:10.1109/TDSC.2020.2984505.

[22] X. Li, Y. Xin, H. Zhu, Y. Yang, Y. Chen, Cross-domain vulnerability detection using graph embedding and domain adaptation, Computers & Security 125 (2023) 103017. doi:10.1016/j.cose.2022.103017.

[23] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Transactions on Software Engineering 49 (8) (2023) 4152–4168. doi:10.1109/TSE.2023.3285910.

[24] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, Journal of Systems and Software 213 (2024) 112038. doi:10.1016/j.jss.2024.112038.

[25] S. Shanbhag, S. Chimalakonda, Exploring the under-explored terrain of non-open-source data for software engineering through the lens of federated learning, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Ideas, Visions and Reflections, ACM, 2022, pp. 1610–1614. doi:10.1145/3540250.3560883.

[26] Y. Yang, X. Hu, Z. Gao, J. Chen, C. Ni, X. Xia, D. Lo, Federated learning for software engineering: A case study of code clone detection and defect prediction, IEEE Transactions on Software Engineering 50 (2) (2024) 296–321. doi:10.1109/TSE.2023.3347898.

[27] C. Zhang, T. Yu, B. Liu, Y. Xin, Vulnerability detection based on federated learning, Information and Software Technology 167 (2024) 107371. doi:10.1016/j.infsof.2023.107371.

[28] L. Wang, J. Liu, X. Gao, et al., FedDense: Tackling non-IID graphs via decoupled structure and feature in federated graph learning, in: Database Systems for Advanced Applications (DASFAA), LNCS 15988, Springer, 2026. doi:10.1007/978-981-95-3906-2_22.

[29] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.

[30] C. Dwork, A. Roth, The Algorithmic Foundations of Differential Privacy, Vol. 9 of Foundations and Trends in Theoretical Computer Science, Now Publishers, 2014. doi:10.1561/0400000042.

[31] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, C. Zhang, FedProto: Federated prototype learning across heterogeneous clients, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, 2022, pp. 8432–8440. doi:10.1609/aaai.v36i8.20819.

[32] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories (MSR), ACM, 2020, pp. 508–512. doi:10.1145/3379597.3387501.

[33] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), ACM, 2023, pp. 654–668. doi:10.1145/3607199.3607242.